Blick - Project Documentation

Product concept, functional specification, verified data sources, architecture, privacy, and delivery scope

Document status: Android-first MVP specification — see "Current implementation status" immediately below for what already exists in this repository versus what the rest of this document specifies as still planned.
Updated: 1 August 2026

Current implementation status (as of 1 August 2026)

This document specifies the full intended product. Most of the sections below describe that end-state design in the present tense, as a specification does — they are **not** a claim that all of it already exists. This section is the authoritative summary of what is actually built today; where the two disagree, this section wins.

Validation status of this update, stated plainly up front: a complete local run — `testDebugUnitTest`, `lintDebug`, `assembleDebug`, and `connectedDebugAndroidTest` on a physical Lenovo TB350FU (Android 14) — has now passed in full: all 271 JVM `@Test` functions and all 21 instrumented `@Test` functions, `lintDebug` with 0 errors (42 warnings), and a working debug APK, with the ongoing-notification loop, routine details live-preview, and full routine management additionally exercised manually on that same device.

Getting there took three work sessions of source beyond the earlier 193-JVM-test baseline. First: the FAB restoration, the Routine Details 30-second auto-refresh, the production notification-permission flow, and the `WorkManagerRoutineScheduler` / `RoutineActiveWindowWorker` scheduling-and-notification-loop implementation, plus a corrective audit fixing a device-local-timezone scheduling bug, sharing one notification-availability check across the notifier/worker/UI, making the Routine Details notification-status hint refresh on every lifecycle resume, and replacing the one-routine FAB's "open then block" flow with an in-place explanation dialog (together, 65 new JVM `@Test` functions and 3 new instrumented ones, reaching 258 JVM / 12 instrumented — all still unrun at that point). Then a verification session actually compiled and ran all of it for the first time, which surfaced and fixed three genuine test-suite issues (an invalid Compose-test-API import, a Robolectric runtime-permission default the tests hadn't accounted for, and a `WorkManager` test-timing assumption that could never hold) — and, more importantly, a real production bug no source review had caught: `RoutineActiveWindowWorker` (a `@HiltWorker`) was never actually receiving its injected dependencies, because the `androidx.hilt:hilt-compiler` annotation processor (distinct from `com.google.dagger:hilt-android-compiler`, already used here for `@HiltViewModel`) had never been added — so WorkManager silently fell back to its plain reflection-based factory and crashed on the worker's real constructor every time it was due to run, meaning **enabling a routine never produced the automatic background notification at all**, with nothing surfacing that failure anywhere a user could see it. Fixed by adding the missing processor (see `android/README.md`'s Build section for the full account). A further session added a dedicated `BOOT_COMPLETED` receiver and durable (Room-backed) stale-snapshot storage across process death, with their own tests, reaching a 260 JVM / 21 instrumented total. Most recently, the ongoing notification was reworked to request promotion to Android 16's Live Update surface (see "Requesting a promoted Live Update" under "Active-window scheduling and the 30-second notification loop" below), a Stop action was added to it (see the same section), and a pre-existing device-timezone bug in the routine details screen's "pause today"/"resume today" controls was fixed to agree with the worker's own device-zone definition of "today" — ten further JVM `@Test` functions across these three changes reached 270 JVM / 21 instrumented. Most recently still, the active-window worker's notification-availability check — previously only performed once before the loop began — was extended to re-run on every loop tick, so the loop now stops fetching departures and burning battery/network for the rest of the window as soon as the user disables notifications partway through, rather than continuing until the window's own end; this added one further JVM `@Test` function, bringing the fully verified total to 271 JVM / 21 instrumented, stated above.

Implemented today (Android client + backend):

- Live SL stop search and transport-mode/line/direction discovery during routine setup.
- The routine-creation wizard, backed by Room persistence.
- A foreground routine details/live-preview screen showing **up to two departures total** (not three — see the corrected departure-count language later in this document), which now fetches immediately when the screen becomes active and again automatically about every 30 seconds while it stays visible (lifecycle-`STARTED`), stopping when the screen is no longer visible and restarting with an immediate fetch when it becomes visible again. A manual "Refresh" action remains available alongside this automatic refresh.
- Routine editing (the same wizard, in an edit mode), enable/disable, pause-today / resume-today (with automatic cleanup of an expired pause), and deletion behind an in-screen Material 3 confirmation dialog.
- An always-visible "Add routine" floating action button on the routine list screen, shown whether the list is empty or already has a saved routine, opening the same creation flow either way.
- A first-beta one-routine limit enforced at the application/UI level: the "Add routine" FAB always stays visible, but with one routine already saved, tapping it shows an in-place dialog explaining the beta limit and pointing at editing/deleting the existing routine, rather than opening a creation flow that could never actually save. `RoutineCreateViewModel.oneRoutineLimitReached` still blocks saving a second routine directly, kept as defence in depth for any other entry point into that screen (e.g. a deep link), but the list screen itself never navigates there once a routine exists.
- Loading, live, no-departures, offline, stale, and unavailable states for the departures section — including a fix ensuring that editing a routine to a different site/line/direction/transport-mode, followed by that new configuration's first fetch failing, can never surface the *previous* configuration's departures mislabelled as "stale" data for the new one. The client's retained last-successful snapshot is now scoped to the exact departure identity (site, line, direction, transport mode) that produced it, and is discarded rather than reused across an identity change.
- The backend's full contract, request validation, upstream normalization, and caching logic (183 passing automated tests as of this update).
- The ongoing-notification foundation (a pure mapper from a routine + the live-departures engine's state + the current time to a notification presentation model, recomputing each departure's countdown rather than trusting a cached value; a real `NotificationCompat`-based builder with one quiet `IMPORTANCE_LOW` channel, one stable notification id reused on every update, a `BigTextStyle` body listing both departures plus a short critical-text countdown for the soonest one, distinct wording for the live/stale/no-departures/offline/unavailable/loading states; and tap-to-reopen-the-routine navigation) **now runs automatically**, not only through the debug-only manual trigger (which remains, in debug builds only, as a development aid): see "Active-window scheduling and the 30-second notification loop" below. The builder also requests promotion (`setRequestPromotedOngoing(true)`) to Android 16's Live Update surface (Samsung's Now Bar where supported) on every post; a `PromotedNotificationChecker` wraps `NotificationManagerCompat.canPostPromotedNotifications()` so the Routine Details debug section can surface whether promotion is actually available on the current device, but the same notification is posted either way — an unsupported or promotion-disabled device automatically falls back to a normal ongoing notification with no separate code path. The notification also always carries a Stop action ("Stop/Unpin" the current window early — same effect as "pause for today" — see "Requesting a promoted Live Update" below).
- Production runtime notification-permission onboarding: a rationale dialog gated behind `AppSettingsDataStore.hasSeenNotificationRationale`, shown at the point the user enables or saves a routine that is meant to show notifications, never repeated after being dismissed once, with a route to the system notification-settings screen when the app's notifications or the Blick channel are disabled. The Routine Details screen also shows a read-only notification-status line (active vs. disabled) computed via the

same shared `NotificationAvailabilityChecker` the notifier and the scheduling worker both use, so all three always agree — it never claims automatic delivery is active when app notifications or the Blick channel are actually off — and that status line now re-checks on every lifecycle resume (e.g. returning from system notification settings), not just once when the screen first loads.

- Active-window scheduling and the 30-second notification loop: saving, editing, enabling, disabling, pausing, resuming, or deleting a routine schedules, replaces, or cancels a `WorkManager` one-time work item for that routine's next active window (unique per-routine work name, so a change can never leave stale scheduled work behind), computed against the device's own local time zone (resolved fresh on every call, so a live device timezone change is picked up without any extra wiring, and a routine's configured wall-clock times are never misinterpreted against UTC or any other fixed zone). When that window starts, a `CoroutineWorker` first checks shared notification availability; only if notifications are actually available does it enter foreground execution, fetch departures immediately, and post or silently update one ongoing, `setOnlyAlertOnce(true)` notification (the same notification id on every update — never a new card) that requests promotion to a lock-screen Live Update, waiting about 30 seconds and repeating until the routine's configured end time. If notifications are unavailable (permission missing, app disabled, or the Blick channel disabled), the worker never enters foreground execution, never starts the loop, and never reports delivery as active — it simply reschedules the next eligible occurrence and exits. On normal completion, a handled failure, or unavailability, the worker re-reads the routine and reschedules its next eligible occurrence (unless the routine has since been deleted or disabled); a genuine cancellation from an edit, delete, or replacement is never treated as a completion and never resurrects the cancelled work. See "Active-window scheduling and the 30-second notification loop" in the architecture section below for exactly how this is built and what its real-world timing limitations are.

Now implemented, beyond the original plan for this milestone:

- A dedicated `BOOT_COMPLETED` receiver (`scheduling/BootCompletedReceiver`), alongside `BlickApplication.onCreate()`'s existing process-start reconciliation and the runtime-registered `ACTION_TIMEZONE_CHANGED` receiver — all three call the same `RoutineScheduleReconciler.reconcileAll()`, which only ever enqueues WorkManager work; it never starts a foreground service directly from the `BOOT_COMPLETED` broadcast itself, so the "no `dataSync` foreground service directly from `BOOT_COMPLETED`" restriction on modern Android (see the architecture section) is still respected — WorkManager decides when the actual foreground worker later runs.
- Persistent stale-data storage: the last successful departure snapshot used for the `Stale` fallback is now Room-backed (`data/local/room/StaleSnapshotEntity.kt`), keyed by routine id and scoped to the exact site/line/direction/mode that produced it, and shared between the routine details screen and the background worker — it survives process death, unlike the previous in-memory-only session scope.

Not yet implemented (described in the sections below purely as the plan):

- Disruptions are not currently displayed anywhere in the Android client. The backend fully implements and tests disruption retrieval and normalization (see "Disruptions" under §8 and §9 below), and the Android side has a matching `Disruption` domain model, DTO, and `DisruptionRepository`/`RemoteDisruptionRepository` — but nothing in the notification, the routine details screen, or any ViewModel calls that repository yet, so no disruption message ever reaches the user today.
- A home-screen widget.
- Exact-time activation — see "Active-window scheduling" below for why this is deliberately best-effort, not exact.
- A production prompt that detects when promoted-notification eligibility isn't enabled and deep-links the user to `Settings.ACTION_APP_NOTIFICATION_PROMOTION_SETTINGS` to turn it on — mirroring the existing notification-permission rationale flow. Today `canPostPromotedNotifications()` is only surfaced in the debug notification section, not acted on anywhere a real user would see it. Note this is Android's own

general per-app control, distinct from and unable to affect Samsung's separate "Live notifications for all apps" developer-only gate — see "Requesting a promoted Live Update" below.

---

1. Project summary

Blick is an Android-first application that automatically displays upcoming SL bus, metro, commuter-train, tram, local-rail, and compatible ferry departures during times chosen by the user.

Instead of requiring the user to open a journey-planning application and search for the same stop every morning, Blick places the relevant information in one quiet, updating Android notification. The user selects an SL site, transport mode, line, direction, active weekdays, and a time window. During that window, the application shows the next relevant departures and disruptions affecting the selection.

The first release is a native Android application. A native iPhone client is planned for a later phase and will use the same platform-neutral backend contract. A Smart TV departure display is a possible future feature, not part of the MVP.

The application is intended to be a calm information display. It does not tell users when to leave, predict whether they will catch a departure, or use stressful messages such as "Leave now" or "Probably too late."

---

2. Problem

Many commuters repeatedly check the same information:

- the same bus stop or station;
- the same line and direction;
- at approximately the same time;
- on the same weekdays.

Existing journey-planning applications usually require several actions: unlock the phone, open an application, search for a stop, station, or journey, and inspect the results. This is unnecessary when the regular commute is already known.

Blick reduces this repeated interaction by showing the information automatically when it is relevant.

---

3. Proposed solution

The user creates a scheduled commute routine, for example:

Setting	Example
Site	Fruängen
Transport	Metro
Line	14
Direction	Toward T-Centralen
Days	Monday-Friday
Active period	07:30-08:00

At 07:30, a quiet, ongoing notification appears:

Fruängen to T-Centralen
Next: 4 min
Then: 11 min / 18 min

The same notification updates during the selected period. When the first departure is no longer relevant, the following departure becomes the next one. At 08:00, the notification disappears and active updating stops.

When there is a relevant disruption, it is added below the departures:

Fruängen to T-Centralen
Next: 4 min
Then: 11 min / 18 min
Disruption: Delays on line 14 due to a signal fault

If no relevant disruption exists, the disruption area is not shown.

---

4. Product principles

Calm

The application presents neutral information without pressure, judgement, repeated alerts, or urgency-based language.

Automatic

Once a routine has been configured, the user should not need to open the application every day.

Relevant

Only departures and disruptions connected to the selected site, transport mode, line, and direction should be displayed.

Glanceable

The most important information must be understandable within a few seconds from the lock screen or notification panel.

Respectful of attention

The application updates one ongoing notification instead of sending a new notification every few minutes.

Honest about data

The application distinguishes scheduled information, real-time information, stale information, and unavailable information. It does not invent missing fields or present old data as current.

---

5. Supported SL transport

The first version is intended to support:

- buses;
- metro;
- commuter trains;

- local and light rail;
- trams;
- SL ferry departures when compatible data is available.

The verified upstream transport-mode values are `BUS`, `METRO`, `TRAM`, `TRAIN`, `SHIP`, `FERRY`, and `TAXI`. The Android UI will expose only modes included in the product scope.

The same routine model applies to each supported mode: the user selects an SL site, a line where relevant, a direction, active days, and an active time window.

---

6. Target users

The initial target users are regular SL commuters who:

- travel from the same site at a similar time on workdays or study days;
- want to check departures while preparing at home;
- prefer passive information over repeatedly searching in a journey planner;
- want relevant disruption information without unrelated network-wide announcements.

The first product is SL-specific. Support for other public-transport operators is a possible later expansion and is not part of the current architecture contract.

---

7. Core user experience

Initial setup

All eight steps below are implemented today; step 8's rationale dialog is shown once, at the point the user first enables or saves a routine meant to show notifications — see "Current implementation status" above.

1. The user opens the Android application.
2. The user searches for and selects an SL site.
3. The user selects a supported transport mode.
4. The user selects a line and direction.
5. The user selects active weekdays.
6. The user selects a start and end time.
7. The user saves the commute routine.
8. The application requests notification permission and explains lock-screen visibility.

Line and direction options are initially discovered from live departures at the selected site. The SL Transport departures endpoint exposes only services inside its current forecast window. A route that is not currently operating may therefore be unavailable during setup. This is a documented MVP limitation. Direction discovery must remain behind an application interface so a more complete source can replace it later without changing the saved-routine model.

Daily operation

Steps 1, 2, 3, 4, 5, and 7 below are implemented in source and verified on a real device — see "Current implementation status" above and the "Active-window scheduling and the 30-second notification loop" architecture note below. Step 6 is not: no disruption information is included anywhere today — see "Not yet implemented" above.

1. The routine becomes active around the configured start time (best-effort, not exact — see the scheduling section below).
2. The application requests current normalized departures from the Blick backend.
3. It filters by the saved site, line, transport mode, and direction code.
4. One ongoing notification appears.
5. The notification is silently refreshed about every 30 seconds during the active period, without a repeated sound, vibration, or extra card.
6. Relevant disruption information is included when available. *(Not yet implemented — see above.)*
7. The notification is removed at the configured end time, and the next eligible occurrence is scheduled.

Separately, and independently of the notification loop above, the Routine Details screen performs its own 30-second automatic refresh purely to keep what's on-screen current while the user is actually looking at that screen — it starts as soon as the screen becomes visible, stops as soon as it isn't, and has no effect on whether the background notification loop is running.

User controls

The notification or application may offer:

- **Refresh**
- **Pause today**
- **End now**
- **Open routine**

These controls should remain optional and visually secondary.

---

8. Functional requirements

Commute routines

The application must allow the user to:

- create a commute routine;
- choose an SL site;
- choose a supported transport mode;
- choose a line and direction;
- choose one or more weekdays;
- set a start and end time;
- enable or disable a routine;
- edit or delete a routine;
- pause a routine for the current day.

The first UI supports one routine. The local database is designed for multiple routines so later expansion does not require replacing the storage model.

Departure display

During an active routine, the application must:

- retrieve current departures through the versioned backend API;
- display up to two upcoming departures total, when available;

- show the selected site, line, and direction;
- calculate the visible countdown locally;
- automatically remove departures whose effective time has passed;
- update the existing notification rather than creating repeated notifications;
- show when the source response was fetched when data may be stale;
- distinguish real-time and scheduled-only information;
- show cancellations when they can be determined from verified upstream fields;
- handle missing, malformed, or unavailable data safely.

Disruptions

The application must:

- retrieve detailed SL disruption messages through the backend;
- match disruptions using the same SL site and line namespace as departures;
- select a Swedish message variant when available;
- display relevant disruptions in neutral language;
- avoid unrelated network-wide messages;
- show cancellations, changed routes, station closures, and significant delays when provided;
- hide the disruption section when nothing relevant is reported.

Notification and lock screen

The notification foundation (mapper, builder, tap navigation), the production permission-rationale flow, and the automatic active-window scheduling/posting/removal loop are all implemented in source and verified — see "Current implementation status" above for the full local run (JVM unit tests, `lintDebug`*,* `assembleDebug`*, and* `connectedDebugAndroidTest`*) plus manual exercising on a physical device, covering the ongoing-notification loop, mid-window edits, and full routine management. Promotion to Android 16's Live Update surface has been confirmed possible on a real Samsung Galaxy S23 Ultra, but only after manually enabling a Samsung-specific Developer options flag — Samsung currently blocks third-party Now Bar access for ordinary users on both One UI 8 and 8.5, with no official removal date found, so a regular user on that same real Android 16 device sees no promoted card by default today, through no fault of Blick's own implementation, and this should be treated as device/firmware-dependent rather than a guaranteed feature; see "Requesting a promoted Live Update" below for the full account.*

The application must:

- request Android notification permission;
- use one ongoing notification during the active period;
- permit full departure information on the lock screen when allowed by the user's system settings;
- remove the notification when the routine ends;
- explain that Android and the user's privacy settings control lock-screen visibility.

The intended presentation is not merely an ordinary notification-drawer entry: while a routine is active, the application must request promotion of that ongoing notification to Android 16's Live Update system surface — a prominent, persistent card intended for the lock screen and, where supported, Samsung's Now Bar — rather than treating an unpromoted ongoing notification as the finished feature. Concretely, the application must:

- declare the `android.permission.POST_PROMOTED_NOTIFICATIONS` manifest permission (a normal, non-runtime permission, separate from the runtime notification permission above);

- request promotion on the routine's ongoing notification (`setRequestPromotedOngoing`), which requires an ongoing notification with a title and a system-supported style — no custom `RemoteViews`;
- check whether the current device and user settings actually allow a promoted post (`canPostPromotedNotifications`) and surface that status for verification rather than assuming promotion always succeeds;
- fall back automatically to a standard ongoing notification — the same content, same update cadence, same removal behavior — on devices or configurations where promotion is unsupported or disabled, with no separate code path and no degraded content;
- provide a Stop/Unpin action on the notification itself, stopping the current active window early (the same effect as "pause for today") without requiring the app to be opened.

The exact visual treatment of a promoted Live Update (whether it renders as a card, where exactly it appears, and any OEM-specific eligibility rules) is controlled by Android and, on Samsung devices, One UI — Blick requests promotion but cannot guarantee that every device grants it.

Future widget

A home-screen widget is outside the first MVP. A later Android version may add a widget that displays the saved route and next departures, provides manual refresh, and opens the routine when tapped.

---

9. Verified data sources

SL Transport API

SL Transport is the primary source for:

- the complete list of SL sites;
- site coordinates and child stop-area IDs;
- stable SL line IDs;
- directions and direction codes;
- scheduled and expected departure times;
- stop-area and stop-point information;
- journey and departure states;
- embedded trip and site deviation signals.

The backend uses:

- `GET /v1/sites?expand=true`
- `GET /v1/sites/{siteId}/departures`

The API is currently keyless. Fair-use requirements still apply.

Official documentation: SL Transport

SL Deviations API

SL Deviations provides detailed disruption records, including:

- stable deviation-case IDs;
- created and modified timestamps;
- publication windows;
- priority fields used for sorting;

- Swedish and other message variants;
- affected stop areas and lines.

The backend uses:

- `GET /v1/messages`

SL Transport Site IDs have been verified as accepted by the SL Deviations `site` filter. Line IDs and stop-area IDs also align with their corresponding entities in SL Deviations responses. No Trafiklab Stop Lookup conversion is required. The Deviations service is keyless but its documentation asks consumers to request data no more than once per minute.

Official documentation: SL Deviations

Excluded upstream APIs

Trafiklab Timetables and Trafiklab Stop Lookup are not used by the SL-only MVP. Removing them avoids incompatible ID namespaces and the absence of a stable numeric line ID in the Timetables route model.

Licence and attribution

Before public release, the operator must satisfy the current SL API terms, including any required registration and acceptance. The product must visibly state that its transport information is based on data retrieved from Trafiklab.se. When practicable, the attribution should link to Trafiklab.se.

Recommended attribution:

Based on information from Trafiklab.se

The application must not imply endorsement by or affiliation with SL, Trafiklab, or Samtrafiken. The first version uses its own visual identity and plain transport labels rather than protected logos or copied brand styling.

Official terms: SL API licence

---

10. Technical architecture

Repository

The project uses one repository with separate areas:

```
blick/
  android/
  backend/
  docs/
```

The Android application and backend have separate build systems and tests. `docs/api-contract.md` holds the detailed machine-facing contract. This project documentation remains the product and architecture overview.

Native Android application

The Android client uses:

- Kotlin;
- Jetpack Compose and Material 3;
- MVVM with repository interfaces;
- Hilt for dependency injection;
- Room for commute routines;

- Preferences DataStore only for small application settings;
- Retrofit with kotlinx.serialization for the backend API;
- WorkManager (not AlarmManager — see "Active-window scheduling" below for why) behind a `RoutineScheduler` interface, implemented by `WorkManagerRoutineScheduler`;
- Android notification APIs behind a notifier interface.

Pinned SDK values:

```
compileSdk = 36
targetSdk = 36
minSdk = 26
```

`minSdk = 26` is a product support decision. It is not described as a technical requirement for notification channels.

Build toolchain: Android Gradle Plugin 9.2.1 using AGP's built-in Kotlin (Kotlin 2.3.10, no separate `org.jetbrains.kotlin.android` plugin), Gradle 9.4.1, KSP 2.3.9. The Gradle wrapper (`gradlew`, `gradlew.bat`, `gradle-wrapper.jar`, and `gradle-wrapper.properties` with its distribution SHA-256 checksum) is committed, so a fresh clone can build without a pre-existing local Gradle install. An earlier Android baseline (predating the ongoing-notification milestone) was successfully validated with a real local run — `assembleDebug` produced a debug APK, `lintDebug` completed with 0 errors, and `testDebugUnitTest` passed 117 JVM unit tests — on a machine with JDK 17 and the Android SDK. The notification-foundation implementation that followed, with 193 JVM source-level `@Test` functions, was itself later validated with a fresh local `./gradlew testDebugUnitTest lintDebug assembleDebug` run: `testDebugUnitTest` passed with no failures, `lintDebug` completed with 0 errors/37 warnings, and `assembleDebug` produced a debug APK. (This run also caught and fixed two real issues surfaced only by actually executing the notification code: a test assertion that could false-fail against a departure's own line designation rather than an actual countdown value, and two Android Lint findings — a `MissingPermission` false positive on the already-guarded notification post, and two `context.getString()` calls inside non-composable callbacks that Lint flagged for potential staleness across configuration changes, both resolved by resolving the strings once via `stringResource()` in composable scope.)

The scheduling/active-window/permission-flow milestone added 42 further JVM `@Test` functions and 3 further instrumented Compose UI `@Test` functions beyond the 193-test baseline (reaching 235 JVM / 9 instrumented in source at the time); a subsequent corrective audit session fixing the device-local-timezone bug, the shared notification-availability check, the lifecycle-aware status refresh, and the explicit one-routine dialog added a further 23 JVM `@Test` functions and 3 further instrumented `@Test` functions, reaching 258 JVM / 12 instrumented. All of that was subsequently compiled and run for the first time, together with a further session's `BOOT_COMPLETED` receiver and durable stale-snapshot storage (adding 2 JVM and 9 instrumented tests), reaching 260 JVM / 21 instrumented. The promoted Live Update request, its Stop action, and the pause-today device-timezone fix added 10 further JVM `@Test` functions with no further instrumented ones, reaching 270 JVM / 21 instrumented. The active-window worker's notification-availability re-check on every loop tick added 1 further JVM `@Test` function with no further instrumented ones — **271 JVM `@Test` functions and 21 instrumented `@Test` functions now exist in source, and all of them pass.** See `android/README.md`'s Build section for the exact toolchain versions, the up-to-date per-file test breakdown, and the real issues that first full run found and fixed (including a genuine production bug, not just test-suite corrections), and see "Validation status of this update" at the top of this document for the summary.

Backend

The backend uses:

- TypeScript;
- Hono;
- Zod validation;

- a Vercel-compatible handler;
- a portable application core shared with a local Node development server;
- explicit upstream-client, normalization, cache, and error-handling interfaces;
- Vitest for backend tests.

The backend remains necessary even though the chosen upstream APIs are keyless. It provides:

- a stable, versioned contract for Android and a future iPhone client;
- upstream schema isolation;
- validation and normalization;
- caching and request deduplication;
- fair-use protection;
- consistent timestamps and errors;
- a controlled place to adapt when upstream services change.

No Trafiklab API key is required by the selected endpoints. `.env.example` should therefore contain only genuinely required runtime configuration, such as local port or environment mode.

Platform-neutral data flow

Stage	Responsibility
Android now / iPhone later	Stores routines, schedules active periods, calculates countdowns, and presents information
Blick backend	Validates requests, fetches and normalizes upstream data, applies caching, and returns versioned JSON
SL Transport	Supplies sites, departures, line IDs, direction codes, stop data, and embedded deviation signals
SL Deviations	Supplies detailed and time-bounded disruption messages

The backend must not return Android-specific notification, Room, or UI concepts.

---

11. Backend API contract

All routes are versioned under `/api/v1`.

Envelopes

Successful response:

```
{
  "schemaVersion": 1,
  "data": {}
}
```

Error response:

```
{
  "schemaVersion": 1,
  "error": {
    "code": "VALIDATION_ERROR",
```

```
    "message": "A safe, actionable message"
  }
}
```

Supported machine-readable error codes:

```
VALIDATION_ERROR
UPSTREAM_ERROR
UPSTREAM_TIMEOUT
UPSTREAM_RATE_LIMITED
NOT_FOUND
RATE_LIMITED
INTERNAL_ERROR
```

Errors must not expose stack traces, environment values, or unnecessary upstream details.

Health

`GET /api/v1/health`

Returns service status and a backend timestamp.

Cache policy:

```
Cache-Control: no-store
```

Site search

`GET /api/v1/stops/search?query=`

The backend searches a cached snapshot from SL Transport `/v1/sites?expand=true`.

Normalized site:

```
siteId
name
note
lat
lon
stopAreaIds
```

Search requirements:

- trim and validate the query;
- use case-insensitive and diacritic-tolerant matching;
- search site name and nullable note;
- rank exact, prefix, token-prefix, and substring matches in that order;
- use deterministic tie-breaking;
- return no more than 20 results.

Cache policy:

```
Cache-Control: public, s-maxage=3600, stale-while-revalidate=86400
```

Departures

`GET /api/v1/departures?siteId=`

`siteId` is required and must be a valid positive SL Site ID.

Top-level data:

```
fetchedAt
```

```
timeZone
siteId
departures
siteDeviations
```

`timeZone` is always `Europe/Stockholm`. `fetchedAt` records when the backend actually fetched the upstream response, not when a cached response was later served.

Normalized departure:

```
departureId
journey
line
direction
directionCode
destination
via
stopArea
stopPoint
scheduledTime
expectedTime
state
isCancelled
tripDeviations
```

The defensive departure identity is:

```
journey.id + ":" + stopPoint.id + ":" + scheduledTime
```

`expectedTime` is nullable. Clients use `expectedTime` when available and otherwise use `scheduledTime`. The backend never returns `minutesRemaining`; each client calculates it from the effective timestamp and the current device time.

`isCancelled` is true when:

- the departure state is `CANCELLED`; or
- a trip deviation has consequence `CANCELLED`.

Other unfamiliar state strings remain available for forward compatibility and are not reinterpreted as cancellation.

Cache policy:

```
Cache-Control: public, s-maxage=30, stale-while-revalidate=30
```

Disruptions

`GET /api/v1/disruptions?siteId=&lineId;=&transportMode;=&future;=`

`siteId` is required. The remaining filters are optional and validated. `future` defaults to `false`, which limits the normal active-routine request to currently published disruptions. A caller must opt in explicitly when future disruption messages are needed.

Normalized disruption:

```
disruptionId
version
createdAt
modifiedAt
validFrom
validUntil
priority
message
affectedStopAreas
```

```
affectedLines
affectedModes
```

`priority` contains the upstream importance, influence, and urgency levels. These values are sorting hints and are not renamed to `severity`.

`message` contains:

```
header
details
scopeAlias
webLink
language
```

The backend selects the message variant whose language is `sv`. It falls back to the first available variant only when no Swedish variant exists.

`affectedModes` is a documented derived list of unique transport modes from affected lines.

Cache policy:

```
Cache-Control: public, s-maxage=60, stale-while-revalidate=60
```

The public contract does not include an unused lines endpoint. One may be added later only when it has a defined consumer, normalized schema, and tests.

---

12. Local information model

Commute routine

A Room `RoutineEntity` supports multiple records even though the first UI exposes one:

```
id
name
siteId
siteName
transportMode
lineId
lineDesignation
directionCode
destinationLabel
activeDaysMask
startTimeMinutes
endTimeMinutes
enabled
pausedDateEpochDay
```

`siteId`, `lineId`, `transportMode`, and `directionCode` are the stable matching fields. `destinationLabel` is a presentation value and must not be the sole identity — there is no separate `directionLabel` field; direction is represented only by `directionCode`. `activeDaysMask` stores the active weekdays as a bitmask (bit 0 = Monday ... bit 6 = Sunday) rather than a list, and `startTimeMinutes`/`endTimeMinutes` store the active window as minutes-since-midnight. `pausedDateEpochDay` is nullable and holds an epoch-day value only when the routine is paused for a specific date. The entity does not track `createdAt`/`updatedAt` timestamps — routines are identified and ordered by their own fields (currently `name`), not by creation history.

The Room DAO exposes:

```
Flow<List<RoutineEntity>>
getById
upsert
```

```
update
delete
deleteById
```

`upsert` is `@Insert(onConflict = OnConflictStrategy.REPLACE)` — an insert that replaces an existing row sharing the same primary key, rather than a separate merge/patch operation.

Preferences DataStore holds only small application settings such as first-launch state, theme choice, and whether an explanation has been dismissed.

Client domain timestamps

Network DTOs receive ISO 8601 strings with explicit offsets. Android maps them into `java.time.Instant` or `ZonedDateTime` domain values instead of retaining raw strings. A future iPhone client will perform an equivalent native conversion.

---

13. Active-window scheduling and the 30-second notification loop

Implemented in source and verified — see "Current implementation status" above and the "Validation status" note at the top of this document for the full local run, including manual exercising of the active-window scheduling/posting/removal loop on a physical device.

Android limits background activity to protect battery life. Blick does not attempt to run continuously — it schedules work only around each routine's own configured window.

Why WorkManager, and why not an exact alarm

- `PeriodicWorkRequest` was **not** used for the 30-second in-window refresh: WorkManager's periodic work has a 15-minute minimum interval, which cannot represent a 30-second cadence. Instead, a single `OneTimeWorkRequest` per routine activates the next window (`WorkManagerRoutineScheduler.scheduleActivation`), and once that worker starts, it owns its own internal 30-second loop for the duration of that one window.
- `USE_EXACT_ALARM` and `SCHEDULE_EXACT_ALARM` are deliberately **not** used, and there is no exact-alarm permission screen. Start-time activation is therefore best-effort, not exact — WorkManager does not guarantee the worker starts at precisely the configured time, only that it will start at or after it, subject to normal Android battery/Doze/OEM scheduling behavior. This document and the app must not claim exact execution.
- Once activated, the worker calls `setForeground()` with a `CoroutineWorker`, so WorkManager itself manages promoting the work to a foreground service — no separate, manually-started foreground service class was written. See Long-running workers.
- The `dataSync` foreground-service type was chosen (`FOREGROUND_SERVICE_DATA_SYNC` permission, `android:foregroundServiceType="dataSync"` on the merged `SystemForegroundService` manifest entry) since the worker's job — fetching current departures over the network and reflecting them in a notification — is a data-sync operation, not media, location, or camera/mic use. See Foreground service types. Android 15+ limits a `dataSync` foreground service to six total hours in any rolling 24-hour period — routine windows are expected to be short (a typical commute window), which is why the worker stops reliably at the routine's configured end time rather than running indefinitely.

Scheduling, replacing, and cancelling work

Each routine's scheduled activation uses a unique, replaceable WorkManager work name (`ExistingWorkPolicy.REPLACE` against `"routine-active-window-" + routineId`). Saving a new routine, editing an existing one, enabling, disabling, pausing for today, resuming, or deleting a routine all call through the same

`RoutineScheduler.scheduleActivation` / `cancelActivation` interface, so a change can never leave stale, obsolete scheduled work behind for that routine. `NextOccurrenceCalculator` computes the next active window (or "active right now") from the routine's active weekdays and start/end time using `ZonedDateTime`, so the JDK itself resolves daylight-saving-time gaps and overlaps rather than the app guessing. The current instant (an injectable `Clock`) and the zone used to interpret it (an injectable `DeviceZoneProvider`, resolving `ZoneId.systemDefault()` fresh on every call in production) are deliberately separate: a routine's weekdays and start/end time are the device's own local wall-clock values, so combining a zone-less `Clock.instant()` with the device's current zone is what makes a 07:30 Stockholm routine actually activate at 07:30 Stockholm time, in both directions of daylight saving, rather than being silently reinterpreted against the clock's own zone (e.g. UTC). Because the zone is re-resolved on every call rather than captured once, a live device timezone change is picked up automatically the next time a schedule is computed, and is also reconciled immediately via a runtime-registered `ACTION_TIMEZONE_CHANGED` receiver.

The 30-second loop itself

Once a window is active, `RoutineActiveWindowWorker.doWork()`:

1. Loads and validates the routine, and checks whether it has been started late (for example after a delay imposed by the system); if the window has already elapsed, it skips posting entirely and reschedules the next eligible occurrence rather than showing a notification for a window that has already ended.
2. Checks shared notification availability (the same `NotificationAvailabilityChecker` the notifier and the Routine Details status hint use) **before** ever entering foreground execution. If notifications are unavailable — permission missing, app notifications disabled, or the Blick channel specifically disabled — the worker never calls `setForeground()`, never starts the loop, never recreates or modifies a disabled channel, and never reports delivery as active; it simply reschedules the next eligible occurrence and exits. Only once availability is confirmed does it enter foreground execution with a valid Blick notification (channel creation still proceeds normally if the channel is merely missing rather than disabled).
3. Fetches current departures through the existing live-departures engine.
4. Filters and maps them through the existing notification model (up to two departures), and posts or silently updates the routine's one ongoing notification — same notification id every time, `setOnlyAlertOnce(true)`, no repeated sound, vibration, heads-up, or extra card — requesting promotion to a lock-screen Live Update on every post (see "Requesting a promoted Live Update" below).
5. Waits about 30 seconds.
6. Repeats from step 3 until the routine's configured end time, a disable, or a deletion — whichever happens first (re-reading the routine from storage on every iteration so an edit made mid-window takes effect without waiting for the next scheduled activation).
7. Removes that notification, stops, and schedules the next eligible occurrence.

If a network fetch fails after an earlier successful one inside the same window, the existing stale/offline semantics apply (no extra alert; the same notification reflects the last known departures with an appropriate stale indication) and the next 30-second tick retries. That stale snapshot is Room-backed (`StaleSnapshotRepository`), keyed by routine id and scoped to the exact departure identity that produced it, and shared with the routine details screen — it survives process death rather than being scoped to one worker run, and either the worker's or the screen's own successful fetch can serve as the other's fallback.

Terminal handling is deliberately three-way: normal completion, a handled failure (for example building the foreground notification or posting an update throws), and a late start or unavailable notifications all re-read the routine and reschedule its next eligible occurrence (unless the routine has since been deleted or disabled) and clean up the active notification if the loop had actually been entered. A genuine `CancellationException` — from the routine being deleted, disabled, or its unique work being replaced mid-run — is always rethrown rather than caught as a handled failure, so cancellation never gets converted into a success/failure result and never causes

the cancelled work to be rescheduled or resurrected.

Requesting a promoted Live Update

Every post or silent update of the routine's ongoing notification also requests promotion to Android 16's Live Update surface, rather than settling for an ordinary notification-drawer entry:

- `RoutineNotificationBuilder` calls `setRequestPromotedOngoing(true)` on the `NotificationCompat.Builder` for every content state, and declares the manifest permission `android.permission.POST_PROMOTED_NOTIFICATIONS` (normal, granted at install time, distinct from the runtime `POST_NOTIFICATIONS` permission).
- The notification body uses `BigTextStyle` (listing both visible departures) with `setShortCriticalText` set to the soonest departure's countdown, or a cancelled indicator — one of the styles the promoted-notification surface actually supports. No custom `RemoteViews` are used anywhere in the builder, since promoted Live Updates do not support them.
- `PromotedNotificationChecker` (backed by `NotificationManagerCompat.canPostPromotedNotifications()`) reports whether the current device and user settings actually allow a promoted post. This is surfaced through the Routine Details debug section for on-device verification; it is not used to change what gets posted — the same builder call is made either way, so an unsupported device or a user who has disabled promoted notifications simply receives a standard ongoing notification automatically, with no separate fallback code path to keep in sync.
- Because `setRequestPromotedOngoing` was only stabilized in `androidx.core` 1.17.0, and the next stable release (1.19.0) requires `compileSdk` 37 (one major version beyond this project's current 36), the dependency is deliberately held at 1.17.0 rather than the latest available version — see `android/README.md`'s Build section.
- **On a real Android 16 device (Samsung Galaxy S23 Ultra), the promoted card did not appear by default — only after manually enabling Settings → Developer options → "Live notifications for all apps."** Blick's own implementation is standard and correct; this restriction is entirely Samsung's, not Blick's, and not fixable in app code. The official Android docs explicitly note that "OEMs can enforce additional criteria for Live update eligibility," and current, independent reporting confirms third-party Now Bar access remains restricted on **both One UI 8 and One UI 8.5** (this device's actual build, not a beta) unless that developer flag is manually enabled. **An earlier version of this document claimed the restriction was beta-only and would lift once One UI 8 shipped stable — that was tech-press speculation, not an official Samsung statement, and this device (already on 8.5) still requiring the flag disproves it. No official Samsung source for a removal date has been found; treat the Now Bar experience as device/firmware-dependent, not a guaranteed feature, and do not restate a resolution timeline without a real source.** There is no manifest flag, permission, or API call available to a third-party app that enables Samsung's developer option — Blick cannot bypass it — and **ordinary users must never be instructed to enable Developer options** as a workaround. With the flag off (the default for any ordinary user), the app's `setRequestPromotedOngoing(true)` request is simply not honored: the OS posts a plain ongoing notification instead, with no error and nothing in `canPostPromotedNotifications()`'s result visibly distinguishing it from the designed fallback path — this project's debug promotion-status line has not been separately confirmed to reflect Samsung's own gate one way or the other. Once the developer flag was enabled, the promoted card appeared on the lock screen near the routine's configured start time, with the same behavior otherwise exercised through the regular-notification testing below (single notification, ~30-second refresh, up to two departures, survives Blick being swiped from Recent Apps, disappears at the routine's end time). Separately, Android's own docs describe a real, permanent, user-facing settings control for the platform's own Live Update eligibility — `Settings.ACTION_APP_NOTIFICATION_PROMOTION_SETTINGS` — that a production build could deep-link users to (see "Not yet implemented" above); this is a distinct, general Android control that **cannot enable**

Samsung's separate "Live notifications for all apps" developer option, so implementing it would not make Blick's Now Bar card generally available on Samsung devices either way. This project's Android 14 physical test device (Lenovo TB350FU) cannot show the promoted surface at all regardless, since it's an Android 16-only platform feature — that device correctly falls back to a plain ongoing notification instead, the same fallback a regular Samsung Android 16 user currently gets by default.

- Separately from promotion specifically, the full notification/scheduling loop was manually verified end to end on the Samsung Galaxy S23 Ultra in its default (non-promoted) state — the exact experience a regular user has today: exactly one notification is ever shown; it refreshes about every 30 seconds with up to two departures; it appears on the lock screen; it survives Blick being swiped away from Recent Apps; it disappears at the routine's configured end time; disabling an active routine removes its notification and re-enabling one during its active window restores it; reboot recovery (`BootCompletedReceiver`) still works; and tapping the notification while Blick is already open does not create a duplicate screen. This is the coverage that matters for every real user until promotion's rollout gate (above) opens up.
- The notification always carries a Stop action (a plain `NotificationCompat.Action`, not a custom view, so it stays valid on the promoted surface too) fulfilling the spec's "Stop/Unpin" requirement. Its `PendingIntent` targets `StopRoutineNotificationReceiver` — an `exported="false"` `@AndroidEntryPoint` `BroadcastReceiver` only ever triggered by this app's own explicit intent, never a system or cross-app broadcast — which hands off to `StopRoutineNotificationAction`, a small, separately unit-tested class (the same split used for `BootCompletedReceiver` and `RoutineScheduleReconciler`). Stopping today's window early is given exactly the same effect as the existing "pause for today" control: it writes `pausedDate` to today's date and reschedules, which `RoutineActiveWindowWorker`'s own next loop tick already observes and stops on (see "The 30-second loop itself" above) — and the notification is also removed directly, so it disappears immediately on tap rather than up to ~30 seconds later. "Today" here is deliberately resolved from the device's current zone (mirroring the worker's own `zonedNow()`), not a zone-less clock, so this write and the worker's read of it always agree, including right around local midnight. Introducing this surfaced a pre-existing bug in `RoutineDetailsViewModel`'s own "pause today"/"resume today" controls, which had been resolving "today" from a zone-less `LocalDate.now(clock)` — the production `Clock` is `Clock.systemUTC()` (see `di/TimeModule.kt`), so shortly after local midnight in any zone ahead of UTC (e.g. Sweden), "pause today" could pause yesterday instead, disagreeing with the worker's own device-zone break condition. Fixed to resolve "today" the same `DeviceZoneProvider`-based way everywhere in this screen. Verified on the same real Samsung Galaxy S23 Ultra: Stop removes the notification immediately and marks the routine "Paused today"; the notification does not return while paused, including when Stop is tapped directly from the locked screen; and Resume today both clears the pause and restarts the notification immediately if the routine's window is still active.

Reboot and process death

WorkManager persists scheduled work through ordinary process death and device reboot on its own. `BlickApplication.onCreate()` additionally reschedules every enabled saved routine at process start, as an idempotent reconciliation safety net — this is not the primary scheduling path. A dedicated `BootCompletedReceiver` runs that same reconciliation directly on `ACTION_BOOT_COMPLETED`, as a further backstop for a reboot after which the app process never happens to start on its own before a routine's next window would have opened; it only ever enqueues WorkManager work from that receiver and does not start a foreground service directly from `BOOT_COMPLETED` (restricted on modern Android; see Restrictions on background starts).

What remains best-effort, not exact

- Start-time activation is best-effort, as stated above — this is a deliberate, documented trade-off, not an oversight.

- The 30-second refresh cadence inside an active window is enforced by the worker's own `delay()` loop while it holds foreground execution, not by a system-guaranteed timer.

Relevant Android documentation:

- Long-running workers
- Foreground service types
- Restrictions on background starts
- Background work
- Create notifications

---

14. Time, caching, and resilience

Stockholm timestamps

SL Transport emits local Stockholm timestamps without an explicit UTC offset. The backend interprets them using the IANA zone `Europe/Stockholm` and emits ISO 8601 timestamps with an explicit offset.

The conversion must not silently rely on a date library's default daylight-saving-time choice. Tests cover:

- normal winter and summer timestamps;
- both sides of the spring 2026 transition;
- both sides of the autumn 2026 transition;
- the duplicated local hour during the autumn overlap;
- a nonexistent local time during the spring gap.

For an ambiguous autumn timestamp, the backend uses the upstream fetch time and forecast window to select the chronologically plausible occurrence. A malformed or impossible timestamp is handled explicitly rather than silently shifted.

SL Deviations already emits timestamps with offsets; they are validated and preserved.

Serverless caching

The backend uses a `Cache` interface with an in-memory implementation for local development and best-effort per-instance reuse on Vercel.

Important limitations:

- Vercel serverless instances do not share dependable process memory;
- a daily site snapshot in memory is not a guaranteed global daily cache;
- HTTP cache headers are the dependable initial shared layer for public GET responses;
- simultaneous identical requests within one instance should be deduplicated;
- **the SL Deviations one-request-per-minute limit is not currently guaranteed in production, and this is a blocker for public deployment, not a solved problem.** Two compounding gaps: the cache/dedup is per-serverless-instance only (Vercel does not guarantee shared memory across instances), and it is keyed per query combination (site/line/transport-mode/future), so real multi-user traffic spanning many different combinations can exceed one request per minute in aggregate even with the cache working exactly as designed. A shared cache (e.g. Redis) plus a real global rate limiter in front of the SL Deviations call path is required before any significant public traffic — preview/manual-testing-scale usage stays within SL's guidance in practice, but that is not the same as the backend enforcing it.

Stale and unavailable information

When live information cannot be refreshed, the application must not present old data as current. Appropriate states include:

```
Unable to update - last checked 07:42
Scheduled time only
No upcoming departures
```

The backend applies timeouts and consistent error mapping. The Android client retains the last successful response only with a visible stale state and appropriate expiry.

The retained response is scoped to the exact departure identity (site, line, direction, and transport mode) that produced it. If the user edits a routine to a different site/line/direction/mode and that new configuration's first fetch fails, the client must not fall back to the previous configuration's retained response — doing so would mislabel unrelated departures as "stale" data for the new routine. A failed refresh may only fall back to stale data that was captured for the identical configuration currently being fetched.

---

15. Edge cases

The application must account for:

- no upcoming departures;
- expected time missing while scheduled time is available;
- temporarily unavailable real-time information;
- a cancelled departure;
- an unfamiliar future state value;
- a changed destination;
- a changed stop point or platform;
- several stop positions at one site;
- a shortened route;
- a line-wide or mode-wide disruption;
- a disruption without a Swedish message variant;
- the device being offline;
- the phone being restarted;
- notification permission being denied;
- lock-screen content being hidden;
- Android delaying scheduled work;
- a routine spanning midnight;
- daylight-saving-time changes;
- weekday and holiday timetable differences;
- upstream timeouts, malformed data, rate limiting, or service failure;
- a route not appearing during setup because it is outside the current departure forecast.

---

16. Minimum viable product

The MVP contains:

1. Native Android setup application.
2. One routine exposed in the UI.
3. Room persistence designed for multiple routines.
4. SL site search.
5. Bus, metro, commuter-train, and local-rail support.
6. Transport, line, and direction selection from available data.
7. Weekday selection.
8. Configurable start and end time.
9. Up to two departures total when available.
10. One scheduled ongoing notification.
11. Lock-screen visibility subject to Android settings.
12. Relevant SL disruption messages.
13. Cancellation presentation.
14. Automatic removal at the end of the routine.
15. Pause-for-today control.
16. Stale and offline states.
17. Visible Trafiklab attribution.
18. Versioned Hono backend deployed to Vercel.

Outside the MVP:

- user accounts;
- cloud routine synchronization;
- GPS tracking;
- analytics;
- advertising;
- social features;
- arbitrary journey planning;
- iPhone implementation;
- TV implementation;
- casting and device pairing;
- all-Sweden operator coverage;
- walking-time advice;
- predictive "Can I catch it?" messages;
- aggressive alert sounds;
- smartwatch support;
- home-screen widget.

---

17. Planned and possible later versions

Planned: native iPhone client

A native iPhone application is planned after the Android MVP. It will consume the same versioned backend contract and reuse the documented transport identity, filtering, timestamp, and cancellation rules.

The iPhone phase must separately investigate:

- Swift and SwiftUI implementation;
- ActivityKit and Live Activities;
- Apple Push Notification service;
- iOS background-execution constraints;
- local persistence;
- App Store delivery and testing.

The Android notification design must not be assumed to translate directly to iOS.

Possible: Smart TV departure display

A Smart TV display is a possible future feature. The preferred first investigation is a secure web or casting experience using the existing backend, rather than separate applications for every TV operating system.

Potential requirements include:

- a large, remote-friendly departure board;
- secure pairing or a temporary display link;
- explicit consent before sharing a saved routine;
- cross-device configuration without exposing travel routines.

No TV, casting, pairing, or cloud-synchronization code is included now.

Other possible additions

- multiple visible routines;
- separate morning and evening schedules;
- Android home-screen widget;
- holiday and temporary schedule controls;
- improved direction discovery;
- accessibility and theme options;
- Wear OS investigation;
- additional Swedish operators after a separate data-contract review.

---

18. Privacy and security

The first Android application does not require the user's identity or precise location.

MVP privacy choices:

- no account;
- routines stored locally in Room;
- no GPS permission;
- no analytics;
- no advertising identifiers;
- no travel-history profile;

- no cloud synchronization;
- no cross-device sharing.

The backend receives public SL identifiers and filters needed for each request. A saved site and routine can still reveal a travel pattern, so logs must avoid retaining complete routine behaviour or associating requests with an identifiable person unnecessarily.

Security requirements:

- HTTPS only;
- strict query validation;
- safe, consistent errors;
- upstream timeouts;
- request-rate protection;
- no stack traces or configuration details in responses;
- environment-based configuration;
- dependencies kept current;
- platform-neutral contract tests.

Although the current upstream APIs are keyless, the backend must remain capable of protecting credentials if a future upstream service requires them.

---

19. Testing and release requirements

Backend tests

- request validation and error envelopes;
- site-search ranking, Swedish characters, and result limits;
- upstream-to-normalized serialization;
- departure identity;
- nullable expected time;
- cancellation from state and trip deviation;
- unknown state preservation;
- Swedish disruption-message selection and fallback;
- cache expiry and in-flight deduplication;
- timeout and upstream failure mapping;
- Stockholm daylight-saving-time transitions.

Android tests

- Room insert, update, delete, and observation;
- routine enable, disable, and pause behaviour;
- DTO-to-domain timestamp conversion;
- effective-time and countdown calculation;
- filtering by site, line, mode, and direction code;
- ViewModel behaviour where implemented;
- stale and offline presentation;

- notification presentation mapping — a pure, Android-free mapper from a routine and the live-departures engine's state to a notification model (`RoutineNotificationMapperTest`);
- notification construction and posting — the real, built `android.app.Notification` and the real framework `NotificationManager`, exercised via Robolectric (channel importance and idempotency, ongoing/visibility/icon flags, tap-intent routine id, per-state content, the app-wide notifications-disabled case, and the Blick notification channel's own `IMPORTANCE_NONE` case — distinct from the app-wide toggle);
- notification-tap intent consumed exactly once, so an Activity recreation cannot replay an already-handled tap;
- `RoutineDetailsViewModel`'s debug notification trigger, via hand-written fakes.

The tests above, and the tests below, have all since been executed via a fresh local `./gradlew testDebugUnitTest lintDebug assembleDebug connectedDebugAndroidTest` run, with no failures — see "Validation status of this update" at the top of this document.

- Add-routine control: an instrumented Compose UI test (`RoutineListScreenTest`, calling the extracted stateless `RoutineListContent` composable directly, no Hilt needed) proving the FAB exists and opens the creation flow both when the routine list is empty and when it already has a saved routine, and that tapping a saved routine still opens it.
- `RoutineDetailsViewModel`'s 30-second auto-refresh: immediate fetch on first becoming active without a duplicate fetch (since `init` already fetched once), fetch again after ~30 virtual seconds, no `Loading` flash on an automatic tick, cancellation stops further ticks when the screen becomes inactive, an immediate fetch on reactivation, and no duplicate/overlapping refresh loop across repeated activation — all using the existing `StandardTestDispatcher`/virtual-time convention, never a real 30-second wait.
- `NextOccurrenceCalculator`: no active weekday, before/inside/at the boundary of today's window, rolling forward across inactive weekdays and into the following week, the paused-for-today exclusion, every day active, and both sides of the 2027 spring daylight-saving gap and the 2026 autumn overlap.
- `WorkManagerRoutineScheduler`: real `WorkManager` (via `WorkManagerTestInitHelper.initializeTestWorkManager`, not a hand-rolled fake) enqueue, replace-on-reschedule, and cancel behavior for a routine's unique work name.
- `RoutineActiveWindowWorker` (via `TestListenableWorkerBuilder` with a fake worker factory): missing routine id fails cleanly; a routine that no longer exists succeeds without posting; a late start past the window's end skips posting and reschedules; a normal run produces the correct number of 30-second ticks, calls `notifier.remove()` exactly once at the end, and reschedules once; the notification model always carries the correct routine id for tap navigation; the routine being disabled mid-run stops the loop early and reschedules with the routine's new disabled state; the routine being deleted mid-run stops the loop and does **not** reschedule; and a fetch failure after an earlier success in the same window produces a `Stale` notification content, not `Offline`/`Unavailable`.
- Scheduler integration: enabling, disabling, pausing for today, resuming, and deleting a routine from `RoutineDetailsViewModel`, saving a new routine from `RoutineCreateViewModel`, and deleting a routine from `RoutineListViewModel` all call through to a fake `RoutineScheduler`'s `scheduleActivation`/`cancelActivation` as expected.

Release checks

- Android build, lint, and unit tests;
- backend type-check, lint, tests, and production build;
- Vercel health and cache-header checks;
- real-device notification and lock-screen testing;
- scheduling tests across reboot and battery restrictions;

- attribution and terms review;
- no secrets or local environment files committed;
- no unrelated screenshots or generated build output in Git.

---

20. Success criteria

The MVP is successful when a user can configure a routine once and then, on subsequent selected weekdays:

- see the correct site, line, and direction automatically;
- see current upcoming departures in one Android notification;
- see the next departure replace the previous one;
- see relevant disruptions without unrelated messages;
- see cancellations and stale-data states clearly;
- have the notification disappear when the active period ends;
- use the feature without reopening the application each morning.

Technical success includes:

- reliable scheduled activation on supported Android devices;
- reasonable battery use;
- correct timestamp and daylight-saving-time behaviour;
- stable platform-neutral backend responses;
- compliance with upstream fair-use and attribution requirements;
- no unnecessary collection of personal information;
- no dependency on an exposed mobile API key.

---

21. Short project description

Blick is an Android-first scheduled departure display for regular SL commuters. Users choose a site, transport mode, line, direction, weekdays, and time window. During that period, upcoming departures and relevant disruptions appear automatically in one calm, updating Android notification. The platform-neutral backend is designed to support a planned native iPhone client and possible future household displays without expanding the initial MVP.